



AutoSlide AI System

A Project Report Submitted to Fulfill the Requirements of CS289

By Group (2)

Reem Ghanem Rajhi (SID: 4555592)

Retaj Bandar ALsaedi (SID: 4555706)

Judy Eissa (SID: 4555493)

Sara Mohammad Al-johani (SID: 4555479)

Batool Eissa Almutairi (SID: 4554052)

Supervised By

Dr.

Marwah Alaofi

14 May, 2026

Abstract

Preparing lecture slides and study materials manually can take a lot of time for professors, and students may struggle to organize and understand the content. To solve this problem, our project introduces an AI-based educational system called the AutoSlide AI System.

The system allows the professor to upload academic content, such as books or documents. The AI system analyzes the content, extracts topics, and automatically generates lecture slides, quizzes, flashcards, and multilingual explanations. Students can view slides, generate learning materials, and save personal copies for studying.

Our project uses AI and NLP techniques to analyze educational texts, extract important topics, and generate interactive learning materials. The system is designed using Layered Architecture to organize the user interface, system logic, AI processing, and database components.

The proposed system helps professors save time and helps students learn in a more interactive and organized way.

Keywords: Artificial Intelligence; Lecture Slide Generation; Educational Platform; MVC Architecture.

Table of Content

Abstract	2
Chapter 1: Introduction.....	4
1.1 Overview	4
1.2 Client Context	4
1.3 Similar Projects.....	5
1.4 Problem Statement.....	5
Chapter 2: Requirements Elicitation	6
2.1 Methodology	6
2.2 Requirements Listing.....	7
Chapter 3: System Models	9
3.1 Use Case Model	9
3.2 Object Model.....	13
3.3 Dynamic Model	15
Chapter 4: Architecture Models & Design Patterns.....	20
4.1 Architecture Models.....	20
4.2 Design Patterns.....	22
Chapter 5: Test Cases and Test Plans	24
5.1 Test Cases	24
5.2 Test Plans.....	25
Chapter 6: Conclusion	28
6.1 Conclusion.....	28
6.2 Lessons Learnt	29
References	30

Chapter 1: Introduction

1.1 Overview

This project introduces a smart NLP-based platform designed to make it easier for professors and students to manage and share educational content. The Professor can upload books, and the system helps organize the content by identifying the main topics and turning them into clear, structured slides. When the professor updates the content, the changes automatically appear for students without needing to download new versions.

Students can also keep their own copies of the topics, make edits, and use helpful study features like Quizzes, flashcards, and explanations in different languages to understand the content.

1.2 Client Context

The client is a university-level educational institution that focuses primarily on both the needs of professors and students in delivering and accessing academic content. The Professor is responsible for preparing lecture materials in slides form, typically by extracting what is relevant and important from textbooks and documents. Students, on the other hand, rely heavily on these slides to study and understand the course.

Currently, the process of preparing these materials is largely manual and time-consuming. The Professor must spend a lot of effort summarizing books, identifying important topics in them, and creating lecture slides, which can lead to inconsistencies in the quality presented to students and can also lead to confusion and frustration among students.

Integrating AI tools can make this process much easier by automatically organizing content, creating clear lecture slides, and updating materials instantly. It can also

make studying for students easier by offering tools like flashcards, quizzes, and explanations in different languages.

Natural Language Processing (NLP) enables systems to analyze, interpret, and extract meaningful information from educational text in order to generate organized learning materials[3].

NLP facilitates multilingual explanations by converting educational content into different languages while preserving meaning and context[3].

Overall, this will make sharing educational content and creating it in the first place much faster, while improving understanding and creating a more interactive learning experience.

1.3 Similar Projects

NotebookLM is an AI tool that helps users summarize books, find main ideas, and give answers [2]. It works well because it uses sources that users upload, but it lacks some features. For example, professors have limited control over choosing course topics, sharing content, or interacting with the learning process.

Our system improves these features by allowing professors to upload books, pick out topics, and automatically create lecture slides. It also lets them share content with students, update in time, and supports interactive tools such as quizzes, flashcards, and explanations in multiple languages.

1.4 Problem Statement

While many students and professors spend a significant amount of time preprocessing content and summarizing books into slides for academic lectures, it can be a difficult and challenging task that, naturally, does not equally appeal to all. The result is a large gap in the quality of learning materials. As professors may choose to update or rearrange their slides, students are often forced to download the same file again. We need a more intelligent way to simplify this arduous process and hopefully produce better, clearer materials for learning.

Chapter 2: Requirements Elicitation

2.1 Methodology

To ensure the proposed AutoSlide AI platform aligns with real-world academic needs and technical constraints, we adopted a scenario-based requirements gathering approach. Specifically, we conducted a Simulated Client Interview to bridge the gap between abstract project goals and concrete technical specifications. By establishing a detailed "Client Persona" representing a stakeholder focused on technical efficiency, security, and compliance, we were able to execute a structured Q&A session.

The following screenshots demonstrate how qualitative goals were gathered during the AI-simulated interview and were translated into the Functional and Non-Functional requirements documented in this report.

Vision & Goals

What is the main goal of this platform?

-To make the learning process easier for both the professors and students.

What problem are you trying to solve for professors and students?

-Professors spend a lot of time turning content into slides, and it is not always easy or consistent. When slides get updated, students have to download them again. Our project makes this process faster, easier, and smarter.

What would success look like for this project?

-A working website with all of its functionalities acquired.

Target Users

Who are the primary users?

-Faculty members, staff, and students.

How do they currently manage educational materials?

-They mostly do it themselves with the help of a website that has little resources.

What frustrations do they face with current solutions?

-How hard it is to make good slides that represents the material well, and the process of updating or editing on slides that has already been published is very time consuming.

Functional Requirements

What types of content should professors upload?

-Course materials, such as books or documents.

How should the system generate slides?

-Using Ai with the help of the professor.

What study tools are essential?

-Flash cards, explanations, and quizzes.

Should students be able to edit and personalize content?

-Yes, with their personalized copy of the content.

How should content sharing work between professors and students?

-In an interactive way.

Do you need multilingual support?

-Yes.

Reliability & Availability

How often should the system be available?

-The platform should be available 90% of the time (High Availability). Downtime should be scheduled during off-peak academic hours.

Should users be able to recover previous versions of their work?

-Yes. The system should maintain a Version history for at least the last 3 iterations of a slide edit.

Maintenance & Updates

How often do you expect system updates or improvements?

-Weekly, for minor feature updates and bug fixes.

Should the system allow easy upgrades?

-Yes. The architecture should be able to allow individual components (like the AI generation engine) to be upgraded without taking the entire site offline.

Compliance & Constraints

Are there legal or institutional regulations to follow?

-Must comply with NDMO (Saudi Arabia) data protection standards.

Are there restrictions on storing or processing data?

-Data must be stored on encrypted cloud servers with a Region Lock if required by the university to keep student data within a specific country.

Success Metrics

How will you measure the success of the platform?

-Grade Improvement, aiming for a 15% increase in average test scores for students using the AI-generated flashcards. Also, a 40% reduction in "Content Burnout" reported by professors.

How important is AI accuracy VS speed?

-A hallucination rate of 1%, verified by a "Fact-Checker" AI agent that cross-references all generated slides against the original source documents.

User Experience

How simple should the system be for first-time users?

-Very simple and easy to use. Users should be able to upload content and create slides in just a few steps without confusion.

What actions should be achievable in the shortest time?

-Uploading course materials and generating slides using AI.

Do you prefer a web app, mobile app, or both?

-Both.

Performance Expectations

How fast should slide generation be?

-Few seconds to under a minute depending on content size.

How many users do you expect at the same time?

-The system should support high concurrency especially during peak academic times.

Should the system handle large files?

-Yes.

Security & Privacy

What type of data needs protection?

-Course materials, student data, and user accounts.

Are there specific security standards you want?

-Basic security standards like encryption and secure login.

Should users have different access levels?

-Yes, professors and students should not have the same access level.

Integration & Compatibility

Should the system integrate with existing platforms?

-Yes. To ensure high adoption, the system should integrate with Learning Systems like Canvas and Blackboard and support Google Drive for document imports.

What file formats must be supported?

-PDF, DOCX, and PPTX.

Should users be able to export/download content?

-Yes. Users should be able to export generated slides to PDF or PPTX formats within 30 seconds of the request.

(This interview was conducted with the assistance of AI to simulate a stakeholder environment.)

2.2 Requirements Listing

2.2.1 Functional Requirements

- The system allows users to log in.
- The system allows the professor to upload content.
- The system displays topics for the professor to select.
- The system generates slides automatically.
- The system allows the professor to edit (add, delete, or draw) and save slides.
- The system updates slides for students when the professor saves changes.
- The system displays slides to students.
- Students can save and edit their copies.
- The system allows professors to share slides with students.
- The system generates flashcards.

- The system generates quizzes.
- The system generates multilingual explanations.

2.2.2 Non-Functional Requirements

- Performance

The system must generate slides from uploaded content within 10-30 seconds per document.
- Scalability

The system should support at least 5,000 users at the same time during peak academic periods.
- Availability

The platform should be available 99.9% of the time monthly.
- Reliability

Automatic retries should occur if content processing fails.
- Security

Only authorized users (professors/students) can access shared materials.
- Maintainability

Bugs should be fixable without affecting other system components.
- Usability

Users should be able to upload content within 1 minute and access generated slides easily with minimal clicks.
- Portability

The system must work on web browsers (Chrome, Safari, Edge) and operating systems (Windows, macOS, Linux)
- Interoperability

The system should integrate with learning platforms.
- Compliance

The system must comply with data privacy regulations.
- Backup and Recovery

The system must automatically back up data every 6 hours.
- Localization

The platform must support multiple languages.

Chapter 3: System Models

3.1 Use Case Model

3.1.1 Use Case Descriptions

Table 3.1.1.1 presents the Log In use case description to show how the user logs into the system.

Use Case:	Log in
Primary Actor:	Professor, Student
Goal:	The user (Professor and Student) can log in to the system.
Preconditions:	• The user has a registered account.
Main Success Flow:	1. The user enters their email and password 2. The system validates the input from the user 3. The system logs the user into the system
Alternative Flow:	A1: Invalid input (email or password) 1. The system displays an error message 2. The user can try again
Postconditions:	• The user is logged into the system

Table 3.1.1.1 Log in Use Case Description

Table 3.1.1.2 presents the Upload Content use case description to show how the professor uploads content to the system.

Use Case:	Upload Content
Primary Actor:	Professor
Goal:	The professor uploads academic content to the system so it can be processed automatically.
Preconditions:	• The professor is logged into the system.
Main Success Flow:	1. The professor selects the "upload content" option. 2. The system prompts the professor to choose a file. 3. The professor selects and uploads a file. 4. The system saves and confirms a successful upload.
Alternative Flow:	None
Postconditions:	• The uploaded content is stored successfully

Table 3.1.1.2 Upload Content Use Case Description

Table 3.1.1.3 presents the Generate Slide use case description to show how the system generates the slides from the uploaded content.

Use Case:	Generate Slides automatically
Primary Actor:	Professor
Goal:	Generate slides from selected topics
Preconditions:	• The professor is logged into the system • The professor uploads content
Main Success Flow :	1. The Professor selects "generate slides". 2. The system displays topics. 3. The Professor selects the desired topic. 4. The Professor clicks "Generate". 5. The System sends requests to the AI system. 6. The AI system generates slides. 7. The System saves slides.
Alternative Flow:	None
Postconditions:	• Slides are generated and stored in the system

Table 3.1.1.3 Generate Slide Use Case Description

Table 3.1.1.4 presents the Edit and Save use case description to show how the professor can edit shared slides and how the updated slide shows on all students' copies.

Use Case:	Edit and Save
Primary Actor:	Professor
Goal:	Enable the professor to modify shared content and ensure the updated version is reflected in the student's copies.
Preconditions:	• The professor is logged into the system. • The content has already been created and shared. • Students have access to their copies of the slides.

Main Success Flow:	1. The professor opens the slide. 2. The professor edits the slide. 3. The professor presses the save button. 4. The system stores the updated slide. 5. The system updates the shared version of the slide. 6. The system applies the updated slide to all students' copies. 7. Students access the slide and see the updated version.
Alternative Flow:	None
Postconditions:	• The content is updated and stored in the system. • All students can see the latest version of the slides in their copies.

Table 3.1.1.4 Edit and Save Use Case Description

Table 3.1.1.5 presents the View Slides use case description to show how the student can access slides.

Use Case:	View Slides
Primary Actor:	Student
Goal:	Allow students to display and view slides.
Preconditions:	• The student is logged into the system. • Slides are generated and stored in the system.
Main Success Flow:	1. The student selects a topic. 2. The system retrieves slides associated with the topic chosen. 3. The system displays slides.
Alternative Flow:	A1: No slides available 1. The System shows "No slides found".
Postconditions:	• Slides are displayed to students.

Table 3.1.1.5 View Slides Use Case Description

Table 3.1.1.6 presents the Add Private Note use case description to show how the student can edit their own slides without changing others.

Use Case:	Add Private Note
Primary Actor:	Student
Goal:	The student adds personal notes on shared slides for individual use.
Preconditions:	• The student is logged into the system. • The student is viewing shared slides.
Main Success Flow:	1. The student opens the shared slides. 2. The system displays slides. 3. The student adds a private note to the slides. 4. The student saves the note. 5. The system stores the note in the student's personal space.
Alternative Flow:	None
Postconditions:	• The note is saved in the student's personal space. • The note is visible only to the student. • The shared slides remains unchanged.

Table 3.1.1.6 Add Private Note Use Case Description

Table 3.1.1.7 presents the Generate Flashcards use case description to show the steps on how the students can generate flashcards.

Use Case:	Generate Flashcards
Primary Actor:	Student
Goal:	The student wants to generate flashcards from generated slides.
Preconditions:	• The student is logged into the system. • Slides are available in the system.
Main Success Flow:	1. The student opens the slides. 2. The student selects the option Generate Flashcards. 3. The system analyzes the content of the slides. 4. The system extracts key concepts and important terms from the slides. 5. The system generates flashcards based on the extracted information. 6. The system displays the generated flashcards to the student.
Alternative Flow:	None
Postconditions:	• Flashcards are successfully generated. • The student can review and study using the flashcards

Table 3.1.1.7 Generate Flashcards Use Case Description

Table 3.1.1.8 presents the Generate Quizzes use case description to show the steps on how the students can generate quizzes.

Use Case:	Generate Quizzes
Primary Actor:	Student
Goal:	The student wants to generate quizzes from the generated slides.
Preconditions:	- The student is logged into the system. - Slides are available in the system.
Main Success Flow:	- The student opens the slides. - The student selects the option Generate Quizzes. - The system asks the student to choose the quiz scope. - The student chooses one of the following options (quiz for all topics or quiz for selected topics). - The system analyzes the selected content. - The system generates quiz questions based on the selected content. - The system displays the generated quiz to the student.
Alternative Flow:	None
Postconditions:	- Quiz questions are generated successfully. - The student can answer the quiz.

Table 3.1.1.8 Generate Quizzes Use Case Description

Table 3.1.1.9 presents the Generate multilingual explanations use case description to show the steps on how the students can generate multilingual explanations.

Use Case:	Generate multilingual explanations
Primary Actor:	Student
Goal:	The system provides multilingual explanations of the slides, allowing the student to select a preferred language.
Preconditions:	- The student is logged into the system. - The slides are available in the system.
Main Success Flow:	- The student views the slides. - The student selects the preferred language. - The system uses AI to translate and explain the slides. - The system displays the translated explanation of the slides in the selected language.
Alternative Flow:	None
Postconditions:	The explanation of the slides is displayed in the selected language.

Table 3.1.1.9 Generate multilingual explanations Use Case Description

3.1.2 Use Case Diagram

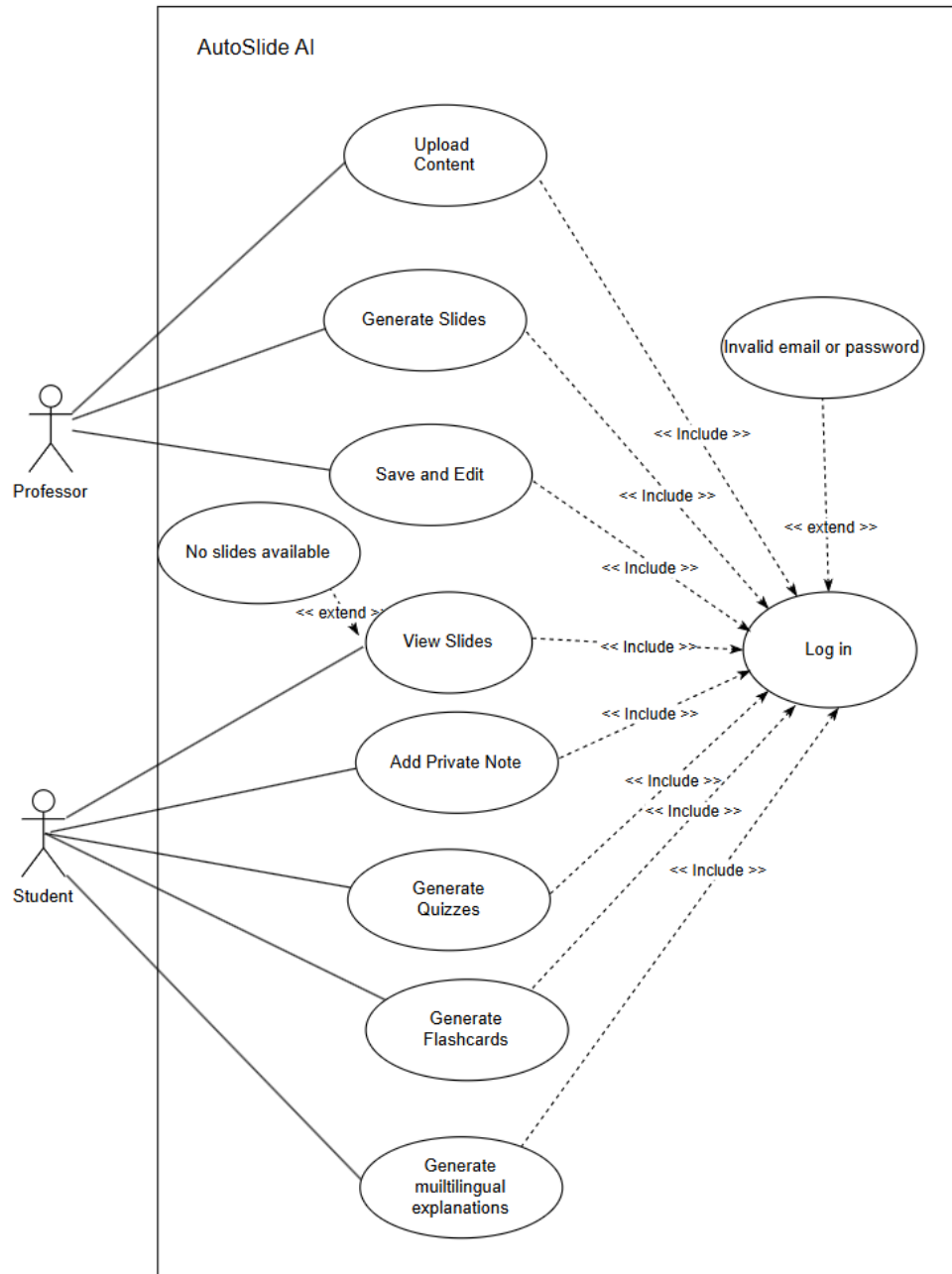


Figure 3.1.2.1 AutoSlide AI System Use Case

3.2 Object Model

3.2.1 Class Diagram

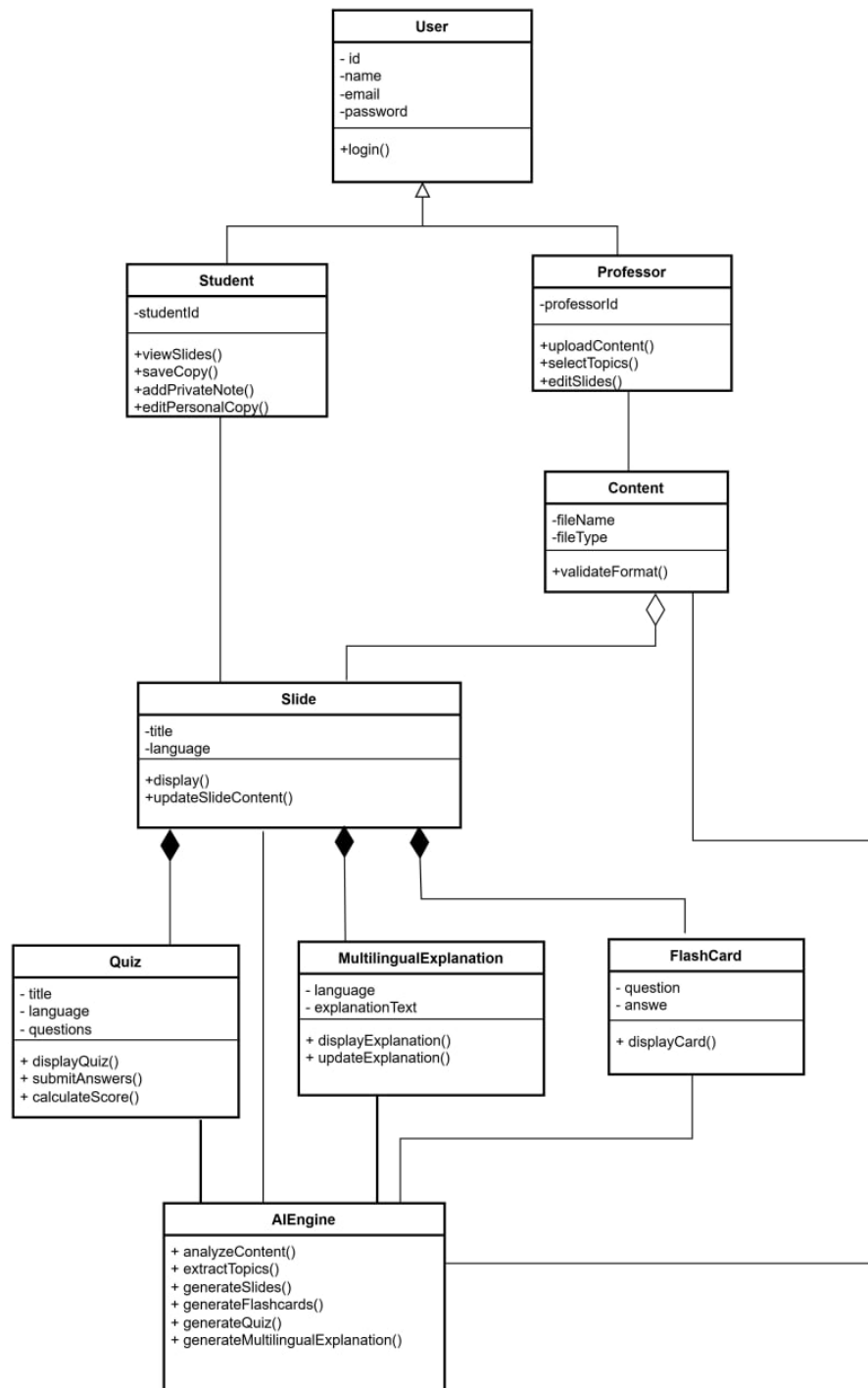


Figure 3.2.1.1 AutoSlide AI System Class Diagram

Figure 3.2.1.1 presents a class diagram that represents the main entities in the AutoSlide AI System and shows the relationships between them.

The **User** class is a general class that stores common user information, such as ID, name, email, and password. Both Student and Professor inherit from User because they share common login information, but have different responsibilities.

The **Student** class represents students who can view slides, save personal copies, add private notes, and edit their personal copies. It has an association relationship with the Slide class, as students interact with slides by viewing them, saving their personal copies, adding notes, and modifying them.

The **Professor** class represents instructors who can upload content, select topics, and edit slides. It is associated with the Content class, since professors are responsible for uploading and managing the academic content used in the system.

The **Content** class represents the uploaded academic materials. It stores information such as the file name and file type. It also includes a function to validate the format. It has an aggregation relationship with the Slide class, where content is used to form and generate slides.

The **Slide** class represents the lecture slides generated by the system. It contains attributes such as title and language, and it includes functions to display and update the slide's content. It has composition relationships with Quiz, FlashCard, and MultilingualExplanation, as these elements are generated from the slide class and depend on it.

The **Quiz** class represents quizzes generated from the slides. It includes attributes such as title, language, and questions. It provides functions to display quizzes, submit answers, and calculate scores. It has a composition relationship with the Slide class, as quizzes are generated from the slide and cannot exist independently without it.

The **FlashCard** class represents flashcards generated from the slides. It contains a question and an answer and includes a function to display the card. It has a composition relationship with the Slide class, as flashcards are generated from the slide class and cannot exist independently without it.

The **MultilingualExplanation** class represents explanations generated in different languages. It contains language and explanation text and includes functions to display and update the explanation. It has a composition relationship with the Slide class, as explanations are generated from the slide class and depend on it.

The **AIEngine** class represents the core intelligence of the system. It is responsible for analyzing content, extracting topics, generating slides, creating quizzes, flashcards, and multilingual explanations. It interacts with Content as input and produces Slide, Quiz, FlashCard, and MultilingualExplanation as outputs.

3.3 Dynamic Model

3.3.1 Sequence Diagrams

We present the sequence diagram in **Figure 3.3.1.1** to show the flow of how the user logs into the system.

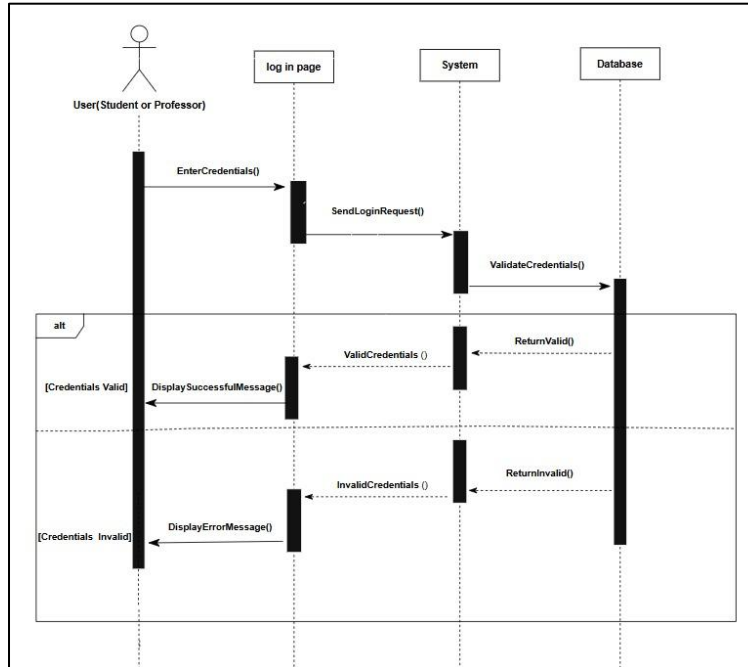


Figure 3.3.1.1 Sequence diagram showing the interactions of Log In use case

We present the sequence diagram in **Figure 3.3.1.2** to show the flow of how the professor uploads content into the system.

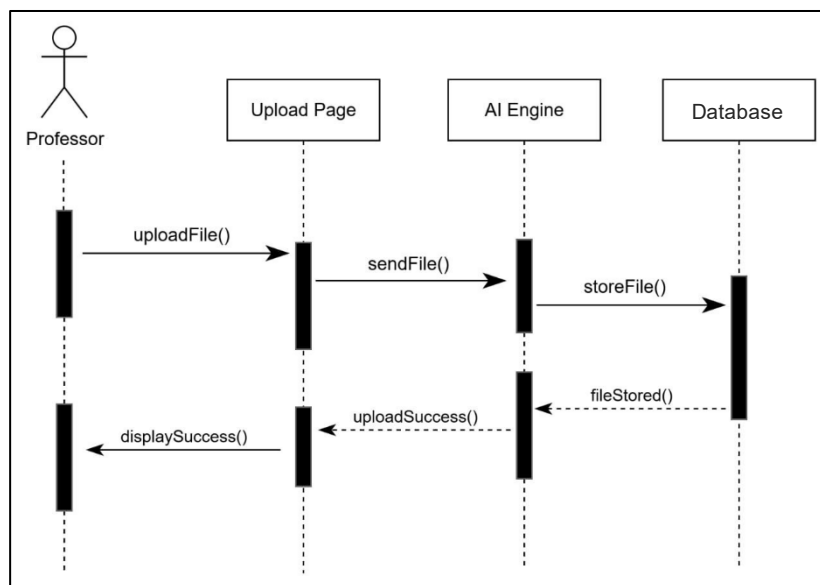


Figure 3.3.1.2 Sequence diagram showing the interactions of Upload Content use case

We present the sequence diagram in **Figure 3.3.1.3** to show the flow of how the professor Generates Slides.

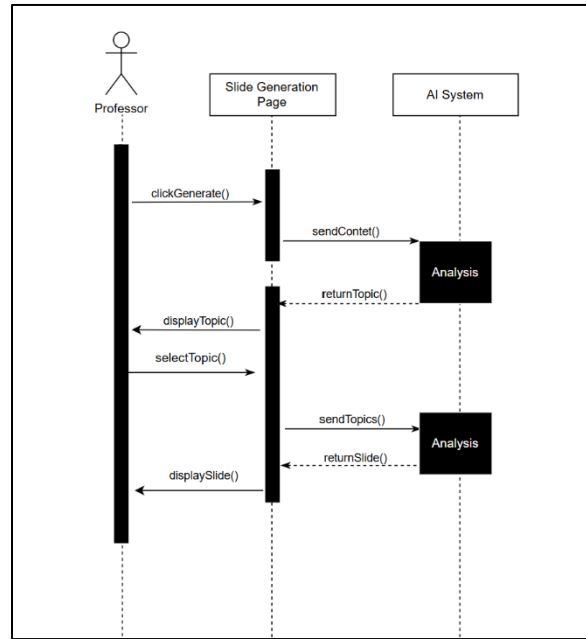


Figure 3.3.1.3 Sequence diagram showing interactions of Generate Slides use case

We present the sequence diagram in **Figure 3.3.1.4** to show the flow of how the system saves the edit from the Professor and updates the students' copies.

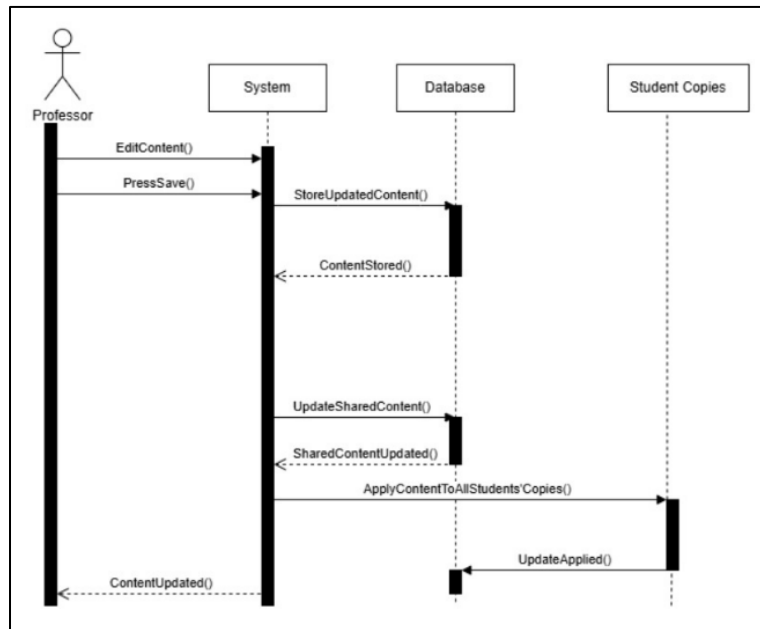


Figure 3.3.1.4 Sequence diagram showing interactions of Edit and Save use case

We present the sequence diagram in **Figure 3.3.1.5** to show the flow of how the Student can view slides.

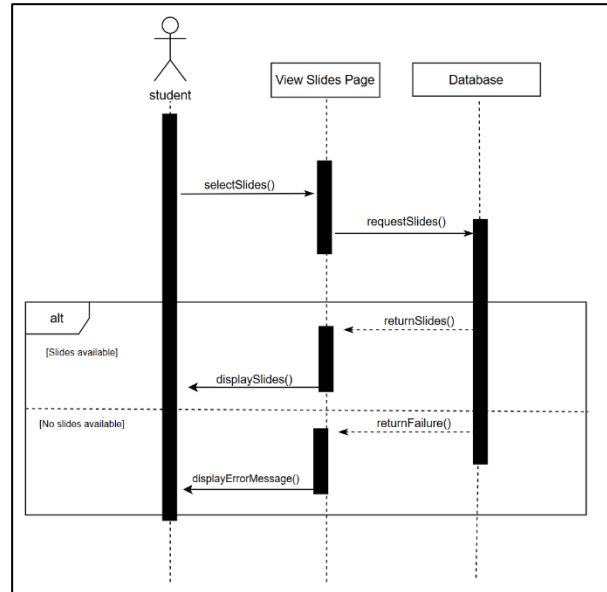


Figure 3.3.1.5 Sequence diagram showing interactions of View Slide use case

We present the sequence diagram in **Figure 3.3.1.6** to show the flow of how the student edits slides.

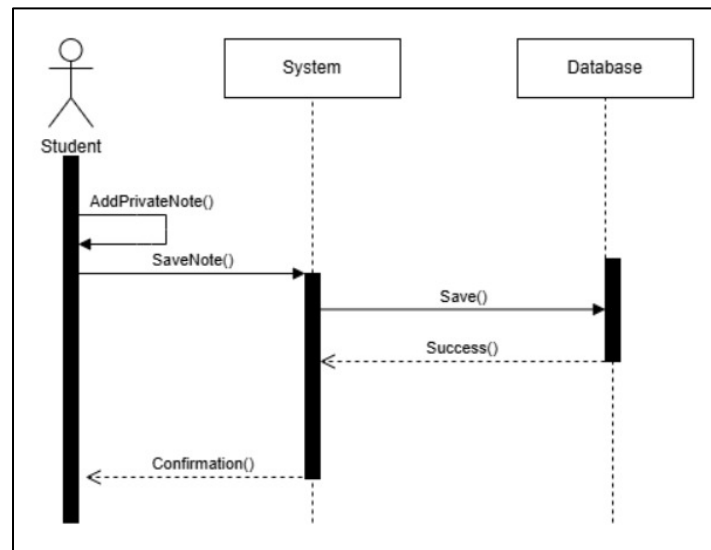


Figure 3.3.1.6 Sequence diagram showing interactions of Add Private Note use case

We present the sequence diagram in **Figure 3.3.1.7** to show the flow of how the student Generates Flashcards.

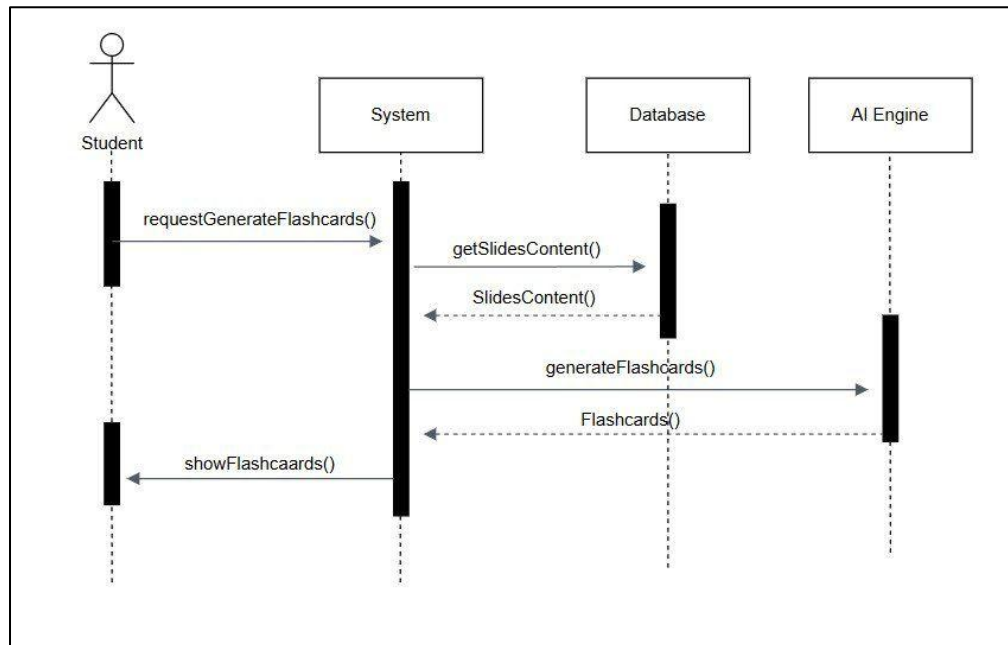


Figure 3.3.1.7 Sequence diagram showing interactions of Generate Flashcards use case

We present the sequence diagram in **Figure 3.3.1.8** to show the flow of how the student Generates Quizzes.

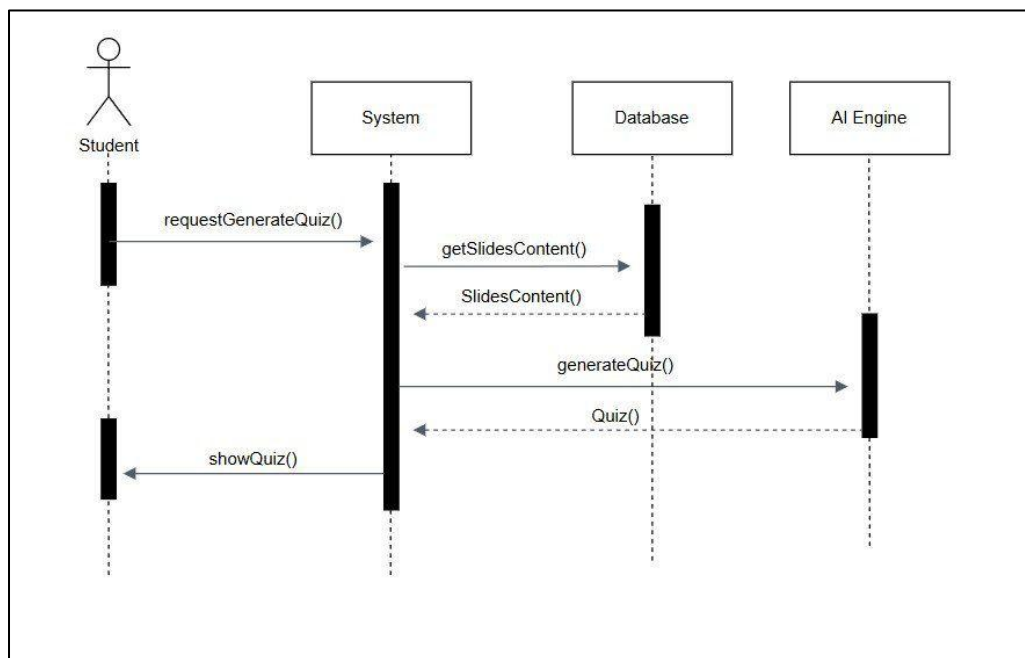


Figure 3.3.1.8 Sequence diagram showing interactions of Generate Quizzes use case

We present the sequence diagram in **Figure 3.3.1.9** to show the flow of how the student Generates Multilingual Explanations.

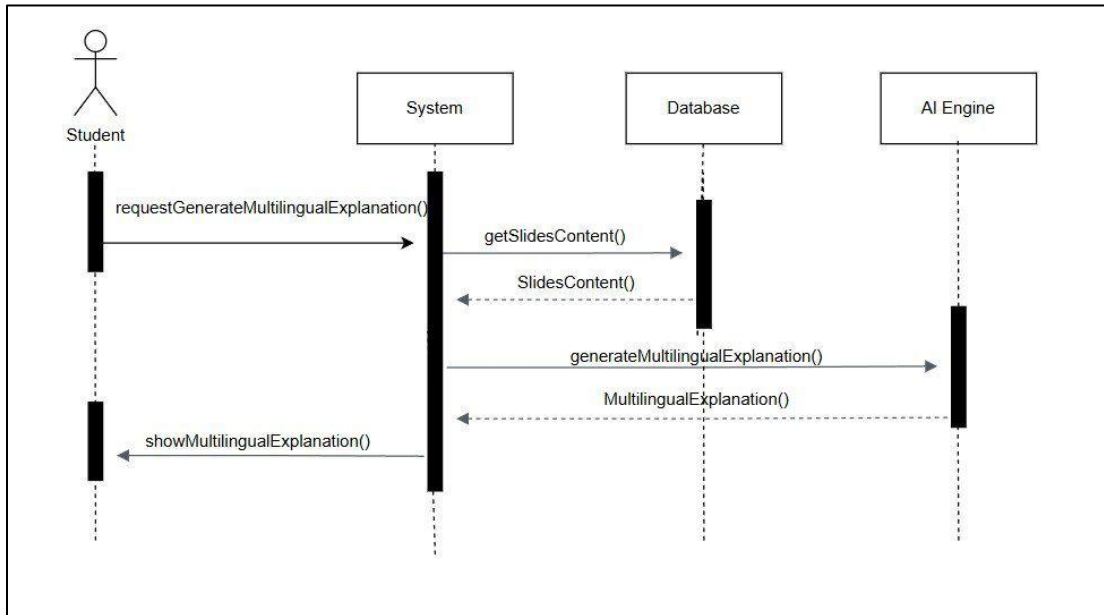


Figure 3.3.1.9 Sequence diagram showing interaction of Generate multilingual explanations use

3.3.2 State Diagram

We present the state diagram **Figure 3.3.2.1** to show the main states of the AutoSlide AI System, starting from uploading academic content, extracting topics, generating and saving slides, sharing them with students, and allowing viewing and updating of slides.

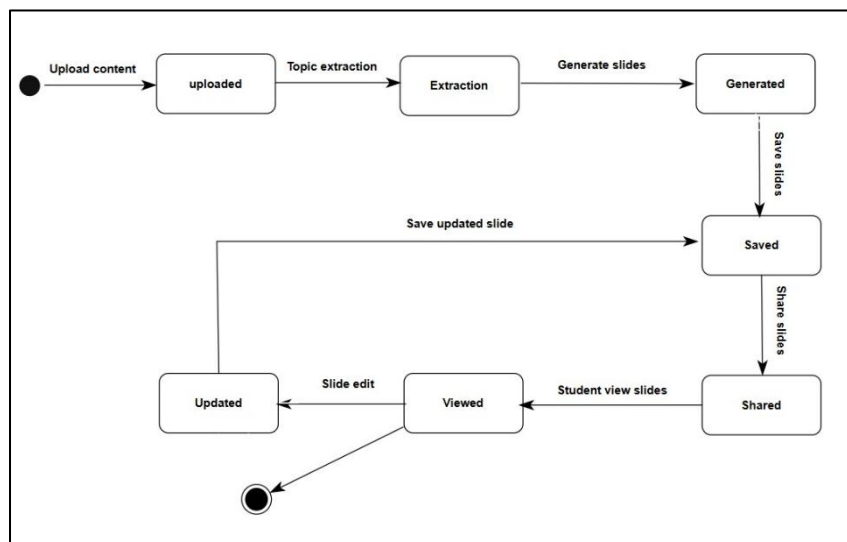


Figure 3.3.2.1 AutoSlide AI System State Diagram

Chapter 4: Architecture Models & Design Patterns

4.1 Architecture Models

The AutoSlide AI System adopts a Layered Architecture as shown in **Figure 4.1.1**, because it separates the system into distinct layers, where each layer has a specific responsibility. This architecture improves organization, maintainability, and scalability. It also allows the AI component to be separated from the user interface and data storage, making the system easier to modify, test, and maintain.

The first layer is the **Presentation Layer**, which represents the user interface used by professors and students. Through this layer, professors can upload academic content, select topics, request the system to generate well-organized slides, edit slides, and share slides with students, as well as log in to the system. Students can view slides, save personal copies, edit their copies, add personal notes, and request the generation of quizzes, flashcards, and multilingual explanations. This layer communicates with the Business Logic Layer by sending user requests.

The second layer is the **Business Logic Layer**, which controls the main system operations. It receives user requests from the Presentation Layer and processes them, including managing uploaded content, validating files, and controlling content sharing. It also coordinates interaction with the AI engine to ensure proper data processing and overall system functionality. Additionally, it handles professor operations such as generating and editing slides, selecting topics, and saving slides, while student operations include viewing slides, creating personal copies, adding notes, and requesting the generation of quizzes, flashcards, and multilingual explanations through the system's AI services.

The third layer is the **AI Layer**, which contains the AI Engine. This layer is responsible for analyzing uploaded content, extracting topics, generating lecture slides, creating quizzes and flashcards, and producing multilingual explanations. This layer communicates with the Business Logic Layer by receiving data for processing and returning generated results.

The final layer is the **Database Layer**, which stores user accounts, uploaded content, generated slides, students' personal copies, and professors' edited copies. This layer communicates with the AI Layer to store and retrieve data, while the Business Logic Layer accesses data indirectly through the AI layer.

In conclusion, the layered architecture is well-suited for the AutoSlide AI System because each layer can be developed and maintained independently. For example, the AI Engine can be improved without changing the user interface, and the database can be updated without affecting the presentation layer. Therefore, the layered architecture supports modularity and helps ensure a more organized and flexible system design.

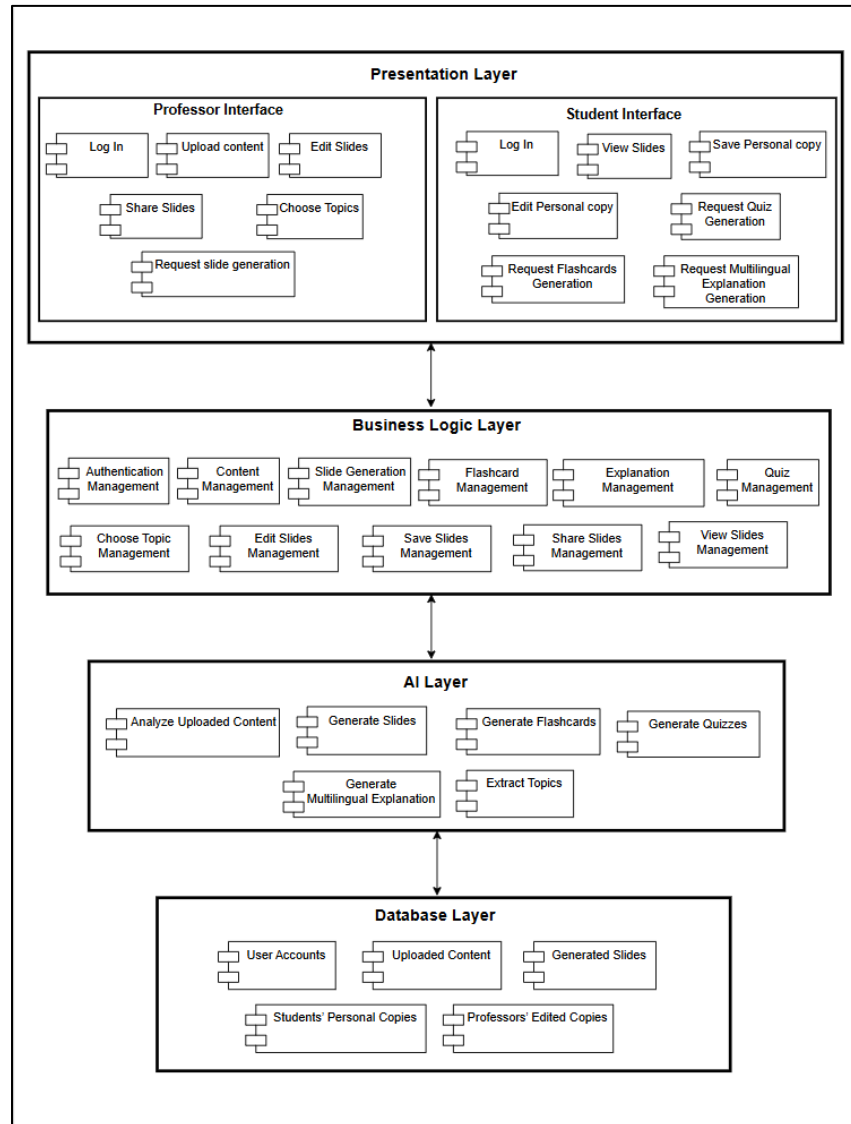


Figure 4.1.1 AutoSlide AI System Layered Architecture

4.2 Design Patterns

MVC (Model-View-Controller) is an architectural pattern used to organize software systems and improve maintainability, scalability, and development efficiency [1]. It separates the system into three components: Model, View, and Controller.

The system architecture is based on the MVC architectural pattern, where the View handles the user interface, the Controller manages user requests and communication between components, and the Model processes data and AI-generated educational content. This separation improves system organization and allows AI functionalities to be integrated efficiently without affecting the user interface or control logic.

This architecture is suitable for an interactive system with a strong focus on user interface and user interaction.

It is also suitable for AI-enhanced solutions because it separates the AI processing logic from the user interface. The Model component can independently handle AI operations such as content analysis, flashcard generation, quiz generation, and slides explanation, while the View remains focused on user interaction and displaying results. This separation improves scalability, maintainability, and future system enhancements.

We present **Figure 4.2.1**. The MVC pattern is used to support interactive educational features that allow users to easily interact with educational content such as lecture slides, flashcards, and quizzes.

The Controller acts as a mediator between the View and the Model.

It handles user requests from both the student and the professor through the system interface.

For professors, it manages actions such as uploading lecture slides to the system.

For students, it processes requests like generating flashcards or quizzes and requesting an explanation of slide content.

It sends the processed results from the Model back to the View for display.

The Model represents the data and interacts with the database. In our system, it is responsible for storing and processing lecture slides, extracting key concepts, and generating learning content such as flashcards, quizzes, and explanations based on the slide content.

The View represents the user interface of the system. In our system, it displays lecture slides and provides interactive options for students, such as generating flashcards, quizzes, and requesting explanations. It also presents the generated results to the user in a clear and organized way.

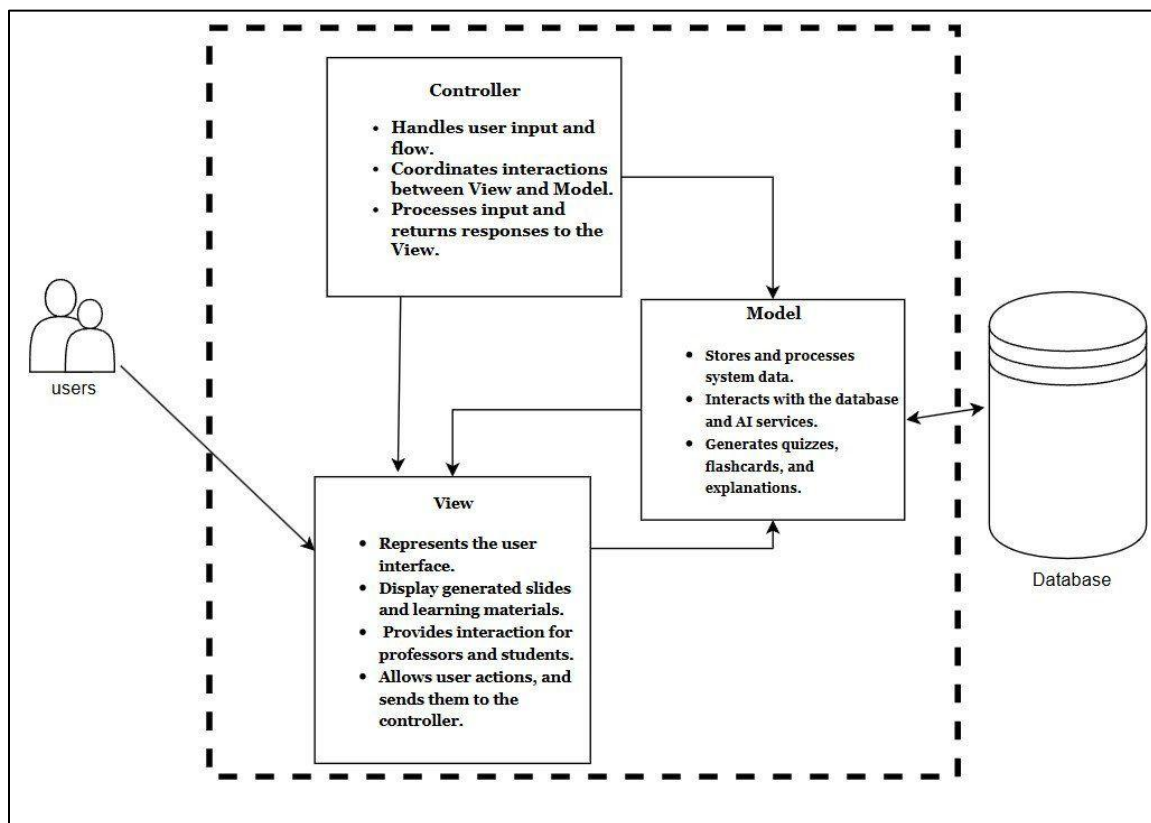


Figure 4.2.1 AutoSlide AI System MVC Pattern

Chapter 5: Test Cases and Test Plans

5.1 Test Cases

1- Functional Testing

Test Case	Multilingual Explanations Function
Objective	Verify that the system provides explanations in the selected language correctly.
Preconditions	• The student is logged in. • Lecture slides are available.
Test Steps	1. Open lecture slides. 2. Select a language. 3. Click "Generate Explanation".
Expected Result	The system displays the explanation in the selected language without errors.

Table 5.1.1 Functional Test

2- Integration Testing

Test Case	Generate Slides Integration
Objective	Verify that the content upload module, topic selection module, AI engine, and database work together correctly to generate slides.
Preconditions	• The professor is logged in. • The content is uploaded. • Topics are available for selection.
Test Steps	1. The Professor opens the uploaded content. 2. The Professor selects the desired topics. 3. The Professor clicks "Generate Slides". 4. The AI engine processes the selected topics. 5. The generated slides are saved in the database.
Expected Result	The system generates slides based on the selected topics, saves them successfully, and displays them to the professor for reviewing or editing.

Table 5.1.2 Integration Test

3- System Testing

Test Case	Complete System Workflow
Objective	Verify that the complete system works correctly from login to content generation and student interaction.
Preconditions	The system is connected to the database, and users are registered.
Test Steps	1. The Professor logs in. 2. The Professor uploads lecture content. 3. The System generates slides. 4. Student logs in. 5. Student views slides. 6. The Student requests quizzes and flashcards. 7. The Student requests multilingual explanations. 8. The Student edits personal notes.
Expected Result	All system functions work correctly together without errors or failures. The generated content is displayed properly, and the student's personal edits do not affect other users

Table 5.1.3 System Test

5.2 Test Plans

Objectives:

The objective of the testing process is to ensure that the AutoSlide AI System operates correctly, reliably, and efficiently while satisfying both the professor's and students' requirements. The testing process aims to verify the accuracy of AI-generated educational content, such as lecture slides, quizzes, flashcards, and multilingual explanations, as well as validate correct communication between system layers and components. In addition, testing helps identify errors, improve system quality, and ensure a smooth and interactive user experience.

Scope:

This test covers the full system workflow, including:

- User login.
- Content upload.

- Slide generation using AI.
- Editing and Sharing slides.
- Viewing slides.
- Adding and Saving personal copies .
- Generating quizzes and flashcards.
- Multilingual explanation.

Methodologies:

- Scenario-based testing (real user scenario).
- Manual testing to simulate actual system usage.

Test Schedule:

- Start Date: Month 1, Week 2
- End Date: Month 1, Week 4
 - Milestones:
 - Test case design completion: Day 5
 - Test environment setup: Day 7
 - Test execution: Day 10
 - Test report submission: Day 14

Tools and Techniques

- Manual Testing.
- Scenario-Based Testing.

Roles and Responsibilities:

- Tester 1: Performs professor actions (upload & generate).
- Tester 2: Performs student actions (view & interact).
- Tester 3: Observes results and records issues.
- Tester 4: Documents test results .

Defect Handling Strategy:

- Record any detected issues during testing.
- Identify the source of the problem.
- Fix the issue.

- Re-run the test to confirm resolution.

Functional Requirements:

Testing verifies features such as content uploading, AI-based slide generation, quizzes, flashcards, multilingual explanations, editing slides, and saving personal copies.

Non-Functional Requirements:

Testing also evaluates usability, performance, reliability, security, scalability, and system response time.

Chapter 6: Conclusion

6.1 Conclusion

This project focused on designing and analyzing an AI-based educational system that helps professors and students interact with educational content more effectively. The platform supports uploading educational materials, generating slides, creating quizzes and flashcards, and providing multilingual explanations using AI techniques. It also enables professors to edit slides and instantly share the updated version with students without requiring them to re-download the content. Students can also create and edit their own personal copies without affecting the original slides. Several UML diagrams and software engineering concepts were applied to model the system structure and user interactions. Furthermore, layered architecture and MVC architecture were used to improve system organization and maintainability. Different testing approaches, including functional, integration, and system testing, were conducted to ensure that the system functions correctly. Overall, the project demonstrates how AI technologies can enhance modern educational systems and improve the learning experience for users.

During the project, several challenges were faced, including organizing system requirements, organizing the required UML diagrams, connecting AI features with system components, designing relationships between classes, and choosing appropriate architecture and design patterns. Additional challenges included managing interactions between different system layers, handling content generation requests correctly, and ensuring clear communication between the AI Engine, database, and user interface. These challenges helped improve our understanding of software engineering concepts, system analysis, and software design principles, while also strengthening our problem-solving and teamwork skills.

6.2 Lessons Learnt

- Learned how to gather functional and non-functional requirements.
- Improved our knowledge of UML diagrams, such as use case, class, sequence, and state.
- Learned how to design functional and non-functional requirements.
- Learned how to apply the MVC architectural pattern in system design.
- Learned how to apply software testing concepts and different techniques.
- Improved our self-learning skills to explore and acquire new knowledge independently.
- Improved communication skills and learned the importance of respecting others' opinions during teamwork.

References

- [1] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill Education, 2019.
- [2] Google, "NotebookLM." [Online]. Available: <https://notebooklm.google/>.
- [3] IBM, "What Is NLP (Natural Language Processing)?" IBM. [Online]. Available: <https://www.ibm.com/think/topics/natural-language-processing>.